

DATA WRANGLING & PREPARATION

| Python Foundations for AI_

MODULE: PANDAS_CORE
STATUS: ONLINE
VERSION: 4.0.1

THE REALITY OF DATA WORK

80%

of a Data Scientist's time is spent **wrangling and cleaning data.**

DATA PREPARATION

MODELING

WRANGLING (80%)

AI (20%)

"Garbage In, Garbage Out."

THE GOLDEN RULE OF AI

MISSION OBJECTIVES



LOAD

Import datasets into pandas DataFrames from various sources.



EXPLORE

Analyze data structure, statistics, and quality issues.



HANDLE

Identify and manage missing values strategically.



CLEAN

Fix data types, remove duplicates, and standardize text.



TRANSFORM

Create new features, filter subsets, and aggregate data.

THE WRANGLING PIPELINE



STEP 01

PANDAS TOOLKIT



STEP 02

LOAD DATA



STEP 03

EXPLORE DATA



STEP 04

HANDLE MISSING



STEP 05

CLEAN DATA



STEP 06

TRANSFORM DATA

YOUR TOOLKIT: PANDAS

DataFrame

A 2-dimensional table with labeled rows and columns. Think "supercharged spreadsheet."

Series

A single column of data. A DataFrame is a collection of Series.

Index

The address of each row (0, 1, 2...). It identifies your data points.

```
import pandas as pd
```

INDEX

	CUSTOMER	PRICE	ITEM
0	Maria	2.50	Cortadito
1	Carlos	3.75	Café con Leche
2	Sofia	5.00	Colada
3	Roberto	2.50	Cortadito
4	Ana	3.75	Café con Leche

SERIES

CUSTOMER	PRICE	ITEM
Maria	2.50	Cortadito
Carlos	3.75	Café con Leche
Sofia	5.00	Colada
Roberto	2.50	Cortadito
Ana	3.75	Café con Leche

DATAFRAME

LOADING DATA INTO PANDAS

The Universal Format: CSV

Comma-Separated Values files are the standard for data exchange. Simple text files where rows are lines and columns are separated by commas.

One Line of Code

pandas can load thousands of rows instantly with a single function call. No manual parsing required.

Other Formats

Excel (.xlsx)

JSON

SQL

Clipboard

```
# The most common way to load data
df =
pd.read_csv('restaurant_orders.csv')
```

```
# Load from a web URL
url =
'https://example.com/data.csv' df =
pd.read_csv(url)
```

```
# Load an Excel file
df =
pd.read_excel('sales_data.xlsx')
```



Watch Out: File paths are case-sensitive!
'Data.csv' is not the same as 'data.csv'.

EXPLORATORY DATA ANALYSIS

> The 5 Essential Commands

`df.shape`

How big is it? (Rows, Columns)

`df.head()`

Show the first 5 rows

`df.info()`

The X-Ray: Types & Missing Data

`df.describe()`

Statistical summary (mean, min, max)

`df.columns`

List of all column names

DATA X-RAY SCAN

```
python3 - df.info()
```

```
# Output of df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   order_id    500 non-null    int64
1   customer    488 non-null    object
2   item        500 non-null    object
3   price       485 non-null    float64
dtypes: float64(1), int64(1), object(2)
memory usage: 15.8+ KB
```

↑ Notice: 'customer' and 'price' have < 500 non-null values.
Data is missing.

DETECTING MISSING VALUES

RAW_DATA_STREAM_V4.CSV

ID	PRICE	QTY	RATING
001	12.50	2	5.0
002	NaN	1	4.0
003	8.99	1	NaN
004	15.00	3	NaN

COUNT MISSING VALUES

```
print(df.isnull().sum())
```

```
order_id    0
price       15
quantity    0
rating      28
dtype: int64
```

CALCULATE PERCENTAGE

```
pct = (df.isnull().sum() / len(df)) * 100
```

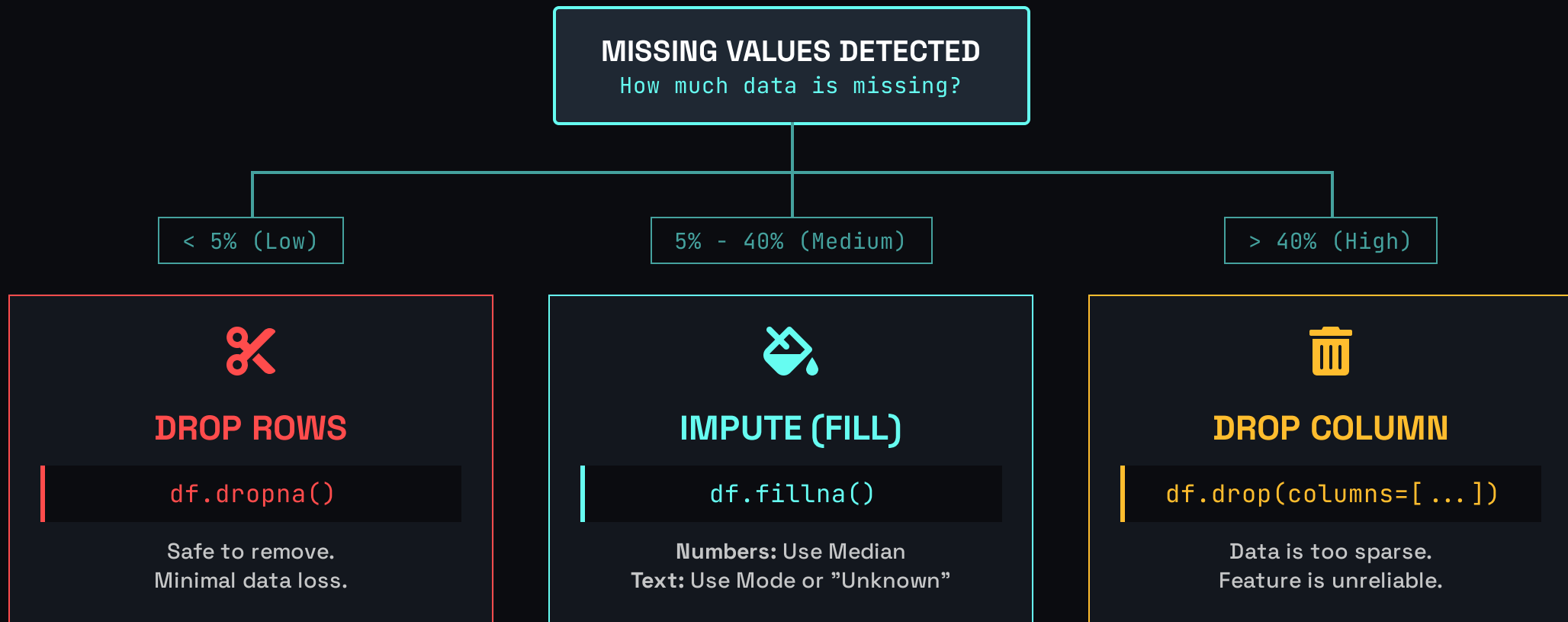
```
price      3.0%
rating     5.6%
```



Context is Critical

Always check percentages. 28 missing rows out of 500 (5.6%) is manageable. 28 out of 50 (56%) is a data crisis.

STRATEGIES FOR MISSING DATA



DATA CLEANING OPERATIONS

📅 FIX TYPES

Convert text strings into real datetime objects to unlock time-based analysis.

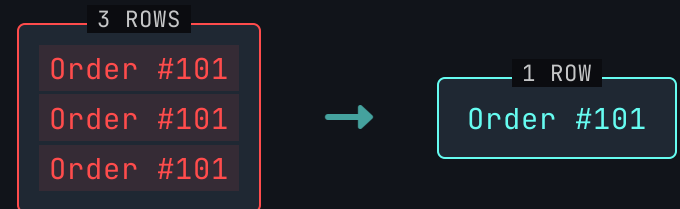
```
df['date'] =  
pd.to_datetime(df['date'])
```



📄 DEDUP

Remove identical rows that skew your analysis counts.

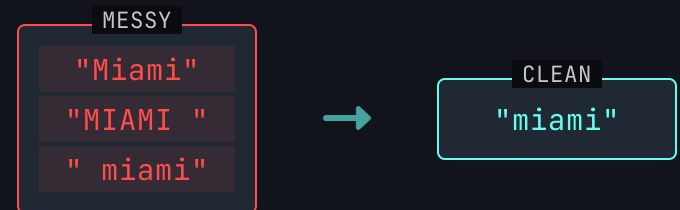
```
df = df.drop_duplicates()
```



A STANDARDIZE

Normalize capitalization and whitespace for consistent grouping.

```
df['city'].str.lower().str.strip()
```



FILTERING & SORTING DATA

🔽 BOOLEAN FILTERING

```
df[df['price'] > 10]
```

⚠️ The Syntax Trap

Python's `and` / `or` do not work in pandas filters. Use bitwise operators: `&` (AND) and `|` (OR). Parentheses are mandatory!

```
df[(cond1) & (cond2)]
```

⬇️ SORTING

```
df.sort_values('price', ascending=False)
```

FILTER: PRICE > 10

ITEM	PRICE
Steak	25.00 ✓
Soda	2.50
Pasta	14.00 ✓
Cookie	3.00
Salad	12.50 ✓

TRANSFORM & GROUPBY

✍️ FEATURE ENGINEERING

```
df['total'] = df['price']  
* df['quantity']
```

📊 THE SPLIT-APPLY-COMBINE PATTERN

```
df.groupby('category')['sales'].mean()
```

1. SPLIT

A: 10
B: 20
A: 30
C: 50
B: 40

2. APPLY (MEAN)

A: 10
A: 30
B: 20
B: 40
C: 50

3. COMBINE

A	20
B	30
C	50

MISSION DEBRIEF



LOAD

Ingest CSV, Excel, and JSON data.



EXPLORE

X-ray data with `.info()` and `.describe()`.



HANDLE MISSING

Drop rows or impute values strategically.



CLEAN

Fix types, duplicates, and messy text.



TRANSFORM

Create features and aggregate with `GroupBy`.

```
├── easy_install-3.4
├── pip
├── pip3
├── pip3.4
├── python -> python3
├── python3 -> /usr/bin/python3
├── include
├── lib
├── python3.4
├── python-wheels
├── lib64 -> lib
└── pyenv.cfg
```

SYSTEM_READY

NEXT MISSION: CHAPTER 05

Data Analysis & Visualization

Now that the data is clean, we start cooking. Get ready to tell stories with data.

